

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 3

«Расчет интеграла функции на заданном промежутке методом Монте-Карло»

Выполнили
студенты

Баянов Равиль Динарович
Кузнецов Даниил Александрович

Группа № Р3434

Преподаватель: Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2025

Цель

Изучить базовые методы Монте-Карло, используемые для интегрирования функций на заданном интервале, и оценить точность интегрирования и основные преимущества и недостатки этих методов.

Задание

Задана функция $f(x) = x^2$. Вычислить интеграл данной функции на интервале $[2, 5]$. Интеграл вычисляется:

1. Аналитически.
2. Простым интегрированием методом Монте-Карло.
3. Интегрированием методом Монте-Карло со стратификацией. Использовать два варианта разбиения: с шагом 1 и с шагом 0.5.
4. Интегрированием методом Монте-Карло с выборкой по значимости. Использовать три варианта плотности вероятности $p_1(x) = x$, $p_2(x) = x^2$, $p_3(x) = x^3$.
5. Интегрированием методом Монте-Карло с многократной выборкой по значимости. Использовать две функции плотности вероятности $p(x) = x$, $p(x) = x^3$. В качестве весов использовать среднюю плотность вероятности (первый вариант расчета $w_1(x) = \frac{p_1(x)}{p_1(x)+p_2(x)}$, $w_2(x) = \frac{p_2(x)}{p_1(x)+p_2(x)}$) и средний квадрат двух плотностей вероятности (второй вариант расчета $w_1(x) = \frac{p_1^2(x)}{p_1^2(x)+p_2^2(x)}$, $w_2(x) = \frac{p_2^2(x)}{p_1^2(x)+p_2^2(x)}$).
6. Интегрированием методом Монте-Карло с использованием русской рулетки. Использовать три точки отсечки: $R_1 = 0.5$, $R_2 = 0.75$, $R_3 = 0.95$.

При вычислении использовать программный генератор псевдослучайных чисел. Использовать разное число выборок при расчете интеграла: $N_1 = 100$, $N_2 = 1000$, $N_3 = 10000$, $N_5 = 100000$.

Результаты представить в виде отчета, в который содержит таблицы значений истинного интеграла, интеграла, вычисленного каждым из методов Монте-Карло, погрешности вычислений и оценки погрешности $\Delta I = \frac{I_{true}}{N}$.

Представит код программы. Язык программирования произвольный.

Аналитическое решение

Используя формулу:

$$\int x^n dx = \frac{x^{n+1}}{n+1}, n \neq -1$$

Вычислим:

$$\int f(x) dx = \frac{x^3}{3} + C, \text{ где } C \in R$$

Теперь вычислим на промежутке:

$$\int_2^5 f(x) dx = \frac{5^3}{3} - \frac{2^3}{3} = 39$$

Ответ: 39

Простое интегрирование методом Монте-Карло

Интеграл от функции равен её среднему значению, умноженному на длину интервала интегрирования.

Алгоритм:

1. Выберем большое количество N случайных точек, равномерно распределенных на интервале интегрирования $[a, b]$
2. Вычислим значение функции $f(x)$ в каждой из этих точек
3. Найдем среднее значение функции по формуле:

$$\bar{f} = \frac{1}{N} \sum_{i=1}^N f(x_i)$$

4. Аппроксимируем интеграл по формуле:

$$\int_a^b f(x) dx \approx (b - a) \bar{f} = (b - a) \cdot \frac{1}{N} \sum_{i=1}^N f(x_i)$$

Интегрирование методом Монте-Карло со стратификацией

Интегрирование методом Монте-Карло со стратификацией — это модификация простого метода Монте-Карло, которая позволяет улучшить точность оценки интеграла за счет деления области интегрирования на более мелкие подобласти (страты) и генерации случайных точек внутри каждой страты.

Алгоритм:

1. Делим область интегрирования на несколько поддиапазонов (страт).
Например, вместо одной большой области можно разбить её на k более мелких, таких как $\left[a, a + \frac{b-a}{k}\right], \left[a + \frac{b-a}{k}, a + 2 \cdot \frac{b-a}{k}\right]$ и т. д.
2. Для каждой отдельной страты генерируем свой набор случайных точек.
3. Вычисляем интеграл для каждой страты отдельно. Затем суммируем результаты интегрирования по всем стратегиям и получаем результат.
4. Полученные значения усредняем и используем формулу для оценки интеграла.

Интегрирование методом Монте-Карло с выборкой по значимости

Интегрирование методом Монте-Карло с выборкой по значимости — это модификация простого метода Монте-Карло, которая позволяет улучшить точность оценки интеграла за счет использования функции плотности вероятности для выборки точек.

Алгоритм:

1. Определяем функцию плотности вероятности $p(x)$, которая аппроксимирует целевую функцию $f(x)$. Функция $p(x)$ должна быть больше или равна нулю в области интегрирования и иметь интеграл равный 1.
2. Сгенерируем N случайных точек x_i из выбранной функции плотности.
3. Для каждой сгенерированной точки x_i вычисляем значение целевой функции $f(x_i)$ и делим его на значение функции плотности $p(x_i)$.
4. Суммируем полученные на предыдущем шаге значения и делим результата на общее кол-во точек N для получения оценки интеграла:

$$I \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}$$

Интегрированием методом Монте-Карло с многократной выборкой по значимости

Интегрирование методом Монте-Карло с многократной выборкой по значимости — это модификация метода Монте-Карло с выборкой по значимости, которая позволяет улучшить точность оценки интеграла за счет использования нескольких функций плотности вероятности для выборки точек.

Алгоритм:

1. Определяем вспомогательное распределение $q(x)$, из которого мы будем брать выборки.
2. Генерируем N независимых случайных точек $x_1, x_2, \dots, x_N$ из вспомогательного распределения $q(x)$.
3. Для каждой точки рассчитаем значение целевой функции $f(x)$ и отношение плотностей вероятности $w(x) = \frac{p(x)}{q(x)}$, где $p(x)$ — это целевое распределение.
4. Рассчитаем интеграл как среднее значение произведения $f(x_i)w(x_i)$ по всем точкам, которое затем умножается на N и делится на N .
5. Оцениваем ошибку с помощью стандартного отклонения выборки, которое также уменьшается с увеличением числа точек.

Интегрированием методом Монте-Карло с использованием русской рулетки

Интегрирование методом Монте-Карло с использованием русской рулетки - это модификация метода Монте-Карло, которая позволяет улучшить точность оценки интеграла за счет использования алгоритма русской рулетки для выборки точек.

Алгоритм:

1. Генерируем случайное число r , равномерно распределенное в интервале $[0, 1]$
2. Обрабатываем с помощью русской рулетки (Повторяем для каждого шага от 1 до N):

- a. Если $r \in [0, R_1)$, прибавьте к сумме $S f(x) \cdot \frac{1}{R_1}$
- b. Если $r \in [R_1, R_2)$, прибавьте к сумме $S f(x) \cdot \frac{1}{R_2}$
- c. Если $r \in [R_2, R_3)$, прибавьте к сумме $S f(x) \cdot \frac{1}{R_3}$
- d. Если $r \in [R_3, 1)$, прибавьте к сумме $S f(x) \cdot \frac{1}{R_3}$

3. Вычисляем значение интеграла по формуле:

$$I \approx \frac{(b-a)}{N} \sum_{i=1}^N f(x_i)$$

4. Вычисляем среднее значение интеграла S по всем итерациям и окончательный результат интеграла I равен $\frac{S}{N} \cdot \frac{b-a}{1}$

Код программы

```
#include <iostream>
#include <random>
#include <iomanip>
#include <cmath>

const std::string EXACT_VALUE_INTEGRAL = "Точное значение интеграла: ";
const std::string SIMPLE_MONTE_CARLO = "Простой метод Монте-Карло";
const std::string STRATIFIED_MONTE_CARLO = "Метод Монте-Карло со стратификацией";
const std::string IMPORTANCE_SAMPLING = "\n=== Метод Монте-Карло с выборкой по значимости ===\n";
const std::string MIS = "\n=== Метод MIS ===\n";
const std::string METHOD_SAMPLING_X = "Выборка по  $p(x) \propto x$ ";
const std::string METHOD_SAMPLING_X2 = "Выборка по  $p(x) \propto x^2$ ";
const std::string METHOD_SAMPLING_X3 = "Выборка по  $p(x) \propto x^3$ ";
const std::string METHOD_MIS_BALANCE = "MIS: средняя плотность ( $w = p/(p+q)$ )";
const std::string METHOD_MIS_SQUARE = "MIS: средний квадрат ( $w = p^2/(p^2+q^2)$ )";
const std::string RUSSIAN_ROULETTE_05 = "Русская рулетка (R=0.5)";
const std::string RUSSIAN_ROULETTE_075 = "Русская рулетка (R=0.75)";
const std::string RUSSIAN_ROULETTE_095 = "Русская рулетка (R=0.95)";

// Функция, которую интегрируем
double f(double x) {
    return x * x;
}

// Метод Монте-Карло для интегрирования
double monteCarloIntegral(double a, double b, int N) {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<double> dis(a, b);

    double sum = 0.0;
    for (int i = 0; i < N; ++i) {
        double x = dis(gen);
        sum += f(x);
    }

    return (b - a) * (sum / N);
}

// Метод Монте-Карло со стратификацией
double stratifiedMonteCarloIntegral(double a, double b, int N, double stride) {
    std::random_device rd;
    std::mt19937 gen(rd());

    double total_integral = 0.0;
    std::vector<double> strata;

    for (double x = a; x < b; x += stride) {
        strata.push_back(x);
    }
}
```

```

    strata.push_back(b);

    int num_strata = strata.size() - 1;
    int points_per_stratum = N / num_strata;
    int total_used = points_per_stratum * num_strata;

    if (total_used < N) {
        points_per_stratum = (N + num_strata - 1) / num_strata;
    }

    for (int k = 0; k < num_strata; ++k) {
        double x_low = strata[k];
        double x_high = strata[k + 1];
        double width = x_high - x_low;

        std::uniform_real_distribution<double> dis(x_low, x_high);
        double sum = 0.0;
        int actual_points = (k == num_strata - 1) ?
            (N - (num_strata - 1) *
points_per_stratum) : points_per_stratum;

        for (int i = 0; i < actual_points; ++i) {
            double x = dis(gen);
            sum += f(x);
        }

        double avg = sum / actual_points;
        total_integral += width * avg;
    }

    return total_integral;
}

// Универсальная функция для тестирования метода интегрирования
void runExperiment(
    const std::vector<int> &N_values,
    const std::function<double(int)> &integrationMethod,
    double exactValue,
    const std::string &methodName) {
    std::cout << "\n=== Метод: " << methodName << " ===\n";
    std::cout << std::setw(8) << "N "
        << std::setw(14) << std::fixed << std::setprecision(6)
<< "Интеграл"
        << std::setw(14) << std::setprecision(6) << " Ошибка" <<
"\n";
    std::cout << std::string(36, '-') << "\n";

    for (int N: N_values) {
        double result = integrationMethod(N);
        double error = std::abs(result - exactValue);

        std::cout << std::setw(8) << N
            << std::setw(14) << std::fixed <<
std::setprecision(6) << result
            << std::setw(14) << std::setprecision(6) << error <<
"\n";
    }
}

```

```

}

// Генерация случайной величины по плотности  $p(x) \propto x^k$  на  $[2, 5]$ 
double sampleFromPk(double a, double b, int k, std::mt19937 &gen) {
    std::uniform_real_distribution<> dis(0.0, 1.0);
    double u = dis(gen);

    double lower_power = std::pow(a, k + 1);
    double upper_power = std::pow(b, k + 1);
    double value = u * (upper_power - lower_power) + lower_power;
    return std::pow(value, 1.0 / (k + 1));
}

// Оценка нормировочной константы  $Z_k = \int_a^b x^k dx$ 
double computeZk(double a, double b, int k) {
    return (std::pow(b, k + 1) - std::pow(a, k + 1)) / (k + 1);
}

// Метод Монте-Карло с выборкой по значимости для  $p(x) \propto x^k$ 
double importanceSamplingIntegral(double a, double b, int N, int k) {
    std::random_device rd;
    std::mt19937 gen(rd());

    double Zk = computeZk(a, b, k); // нормировочная константа
    double sum = 0.0;

    for (int i = 0; i < N; ++i) {
        double x = sampleFromPk(a, b, k, gen);
        double px = std::pow(x, k) / Zk; // плотность  $p(x)$ 
        sum += f(x) / px;
    }

    return sum / N; //  $\approx \int f(x) dx$ 
}

const double Z1 = computeZk(2.0, 5.0, 1);
const double Z3 = computeZk(2.0, 5.0, 3);

// Плотности вероятности (нормированные)
double p1(double x) {
    return x / Z1;
}

double p2(double x) {
    return std::pow(x, 3) / Z3;
}

// MIS с двумя плотностями и двумя типами весов
enum weightType {
    BALANCE, SQUARE
};

double misIntegral(int N, weightType weightType) {
    std::random_device rd;
    std::mt19937 gen(rd());

    int N1 = N / 2;
    int N2 = N - N1;

```

```

double sum1 = 0.0;
double sum2 = 0.0;

// Выборка из  $p_1(x) \propto x$  ( $k=1$ )
for (int i = 0; i < N1; ++i) {
    double x = sampleFromPk(2.0, 5.0, 1, gen);
    double p1_val = p1(x);
    double p2_val = p2(x);

    double w1;
    if (weightType == BALANCE) {
        w1 = p1_val / (p1_val + p2_val);
    } else {
        double p1_sq = p1_val * p1_val;
        double p2_sq = p2_val * p2_val;
        w1 = p1_sq / (p1_sq + p2_sq);
    }

    sum1 += (f(x) / p1_val) * w1;
}

// Выборка из  $p_2(x) \propto x^3$  ( $k=3$ )
for (int i = 0; i < N2; ++i) {
    double x = sampleFromPk(2.0, 5.0, 3, gen);
    double p1_val = p1(x);
    double p2_val = p2(x);

    double w2;
    if (weightType == BALANCE) {
        w2 = p2_val / (p1_val + p2_val);
    } else {
        double p1_sq = p1_val * p1_val;
        double p2_sq = p2_val * p2_val;
        w2 = p2_sq / (p1_sq + p2_sq);
    }

    sum2 += (f(x) / p2_val) * w2;
}
return sum1 / N1 + sum2 / N2;
}

// Генерация равномерной точки на [a, b]
double sampleUniform(double a, double b, std::mt19937 &gen) {
    std::uniform_real_distribution<> dis(0.0, 1.0);
    return a + (b - a) * dis(gen);
}

// Оценка интеграла методом Монте-Карло с русской рулеткой
double russianRouletteIntegral(int N, double continueProb) {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(0.0, 1.0);

    double sum = 0.0;
    double a = 2.0, b = 5.0;
    double p_uniform = 1.0 / (b - a);

    for (int i = 0; i < N; ++i) {

```

```

        double x = sampleUniform(a, b, gen);
        double unweighted_contribution = f(x) / p_uniform;

        // Русская рулетка
        double xi = dis(gen);
        if (xi < continueProb) {
            double weight = 1.0 / continueProb;
            sum += unweighted_contribution * weight;
        }
    }

    return sum / N;
}

int main() {
    const double a = 2.0;
    const double b = 5.0;
    const double exact = 39.0; // Точное значение интеграла

    std::vector<int> N_values = {100, 1000, 10000, 100000};

    // Аналитическое решение
    std::cout << EXACT_VALUE_INTEGRAL << exact << "\n";

    // Простой метод Монте-Карло
    auto simpleMonteCarlo = [a, b](int N) {
        return monteCarloIntegral(a, b, N);
    };
    runExperiment(N_values, simpleMonteCarlo, exact,
SIMPLE_MONTE_CARLO);

    // Метод Монте-Карло со стратификацией
    auto stratifiedMonteCarlo = [a, b, exact](int N) {
        double stride = (b - a) / 10.0;
        return stratifiedMonteCarloIntegral(a, b, N, stride);
    };
    runExperiment(N_values, stratifiedMonteCarlo, exact,
STRATIFIED_MONTE_CARLO);

    // Метод Монте-Карло с выборкой по значимости
    std::cout << IMPORTANCE_SAMPLING;
    // Выборка по значимости:  $p(x) \propto x$ 
    auto impMethod1 = [a, b](int N) {
        return importanceSamplingIntegral(a, b, N, 1);
    };
    runExperiment(N_values, impMethod1, exact, METHOD_SAMPLING_X);

    // Выборка по значимости:  $p(x) \propto x^2$ 
    auto impMethod2 = [a, b](int N) {
        return importanceSamplingIntegral(a, b, N, 2);
    };
    runExperiment(N_values, impMethod2, exact, METHOD_SAMPLING_X2);

    // Выборка по значимости:  $p(x) \propto x^3$ 
    auto impMethod3 = [a, b](int N) {

```

```

    return importanceSamplingIntegral(a, b, N, 3);
};

runExperiment(N_values, impMethod3, exact, METHOD_SAMPLING_X3);

// MIS
std::cout << MIS;
// MIS: средняя плотность (balance heuristic)
auto misBalance = [](int N) {
    return misIntegral(N, BALANCE);
};
runExperiment(N_values, misBalance, exact, METHOD_MIS_BALANCE);

// MIS: средний квадрат (square heuristic)
auto misSquare = [](int N) {
    return misIntegral(N, SQUARE);
};
runExperiment(N_values, misSquare, exact, METHOD_MIS_SQUARE);

// Русская рулетка с тремя порогами
auto rr_05 = [](int N) { return russianRouletteIntegral(N, 0.5);
};
auto rr_075 = [](int N) { return russianRouletteIntegral(N, 0.75);
};
auto rr_095 = [](int N) { return russianRouletteIntegral(N, 0.95);
};

runExperiment(N_values, rr_05, exact, RUSSIAN_ROULETTE_05);
runExperiment(N_values, rr_075, exact, RUSSIAN_ROULETTE_075);
runExperiment(N_values, rr_095, exact, RUSSIAN_ROULETTE_095);

return 0;
}

```

Вывод программы

Точное значение интеграла: 39

=== Метод: Простой метод Монте-Карло ===

N	Интеграл	Ошибка
100	41.340386	2.340386
1000	39.383646	0.383646
10000	39.072857	0.072857
100000	39.029855	0.029855

=== Метод: Метод Монте-Карло со стратификацией ===

N	Интеграл	Ошибка
100	nan	nan
1000	39.083248	0.083248
10000	39.018567	0.018567
100000	39.002903	0.002903

=== Метод Монте-Карло с выборкой по значимости ===

=== Метод: Выборка по $p(x) \propto x$ ===

N	Интеграл	Ошибка
100	38.450918	0.549082
1000	39.311472	0.311472
10000	39.079315	0.079315
100000	39.012771	0.012771

=== Метод: Выборка по $p(x) \propto x^2$ ===

N	Интеграл	Ошибка
100	39.000000	0.000000
1000	39.000000	0.000000
10000	39.000000	0.000000
100000	39.000000	0.000000

=== Метод: Выборка по $p(x) \propto x^3$ ===

N	Интеграл	Ошибка
100	39.057617	0.057617
1000	39.039534	0.039534
10000	38.921916	0.078084
100000	38.999904	0.000096

=== Метод MIS ===

=== Метод: MIS: средняя плотность ($w = p/(p+q)$) ===

N	Интеграл	Ошибка
100	39.105701	0.105701
1000	38.954503	0.045497
10000	38.987438	0.012562
100000	38.997977	0.002023

=== Метод: MIS: средний квадрат ($w = p^2/(p^2+q^2)$) ===

N	Интеграл	Ошибка
100	40.215449	1.215449
1000	38.762036	0.237964
10000	39.056396	0.056396
100000	39.004488	0.004488

=== Метод: Русская рулетка (R=0.5) ===

N	Интеграл	Ошибка
100	44.865118	5.865118
1000	38.978936	0.021064
10000	38.388313	0.611687
100000	39.133450	0.133450

=== Метод: Русская рулетка (R=0.75) ===

N	Интеграл	Ошибка
100	40.142005	1.142005
1000	38.744920	0.255080
10000	39.045741	0.045741
100000	38.970799	0.029201

=== Метод: Русская рулетка (R=0.95) ===

N	Интеграл	Ошибка
100	39.917799	0.917799
1000	39.381253	0.381253
10000	38.956840	0.043160
100000	38.992552	0.007448

